

目录



- [01、RAG 的整体设计](#)
- [02、AST 解析](#)
- [03、Embedding](#)
- [04、向量存储](#)
- [05、混合检索](#)
- [06、代码关系图谱](#)
- [07、集成到 Agent](#)
- [08、CLI 实战](#)
 - [代码库更新了怎么办？](#)
- [ending](#)
- [简历包装：PaiCLI 项目](#)

目录

- [01、RAG 的整体设计](#)
- [02、AST 解析](#)
- [03、Embedding](#)
- [04、向量存储](#)
- [05、混合检索](#)
- [06、代码关系图谱](#)
- [07、集成到 Agent](#)
- [08、CLI 实战](#)
 - [代码库更新了怎么办？](#)
- [ending](#)
- [简历包装：PaiCLI 项目](#)

用 SQLite + Embedding 给 Agent 加上 RAG，从此秒懂项目源码

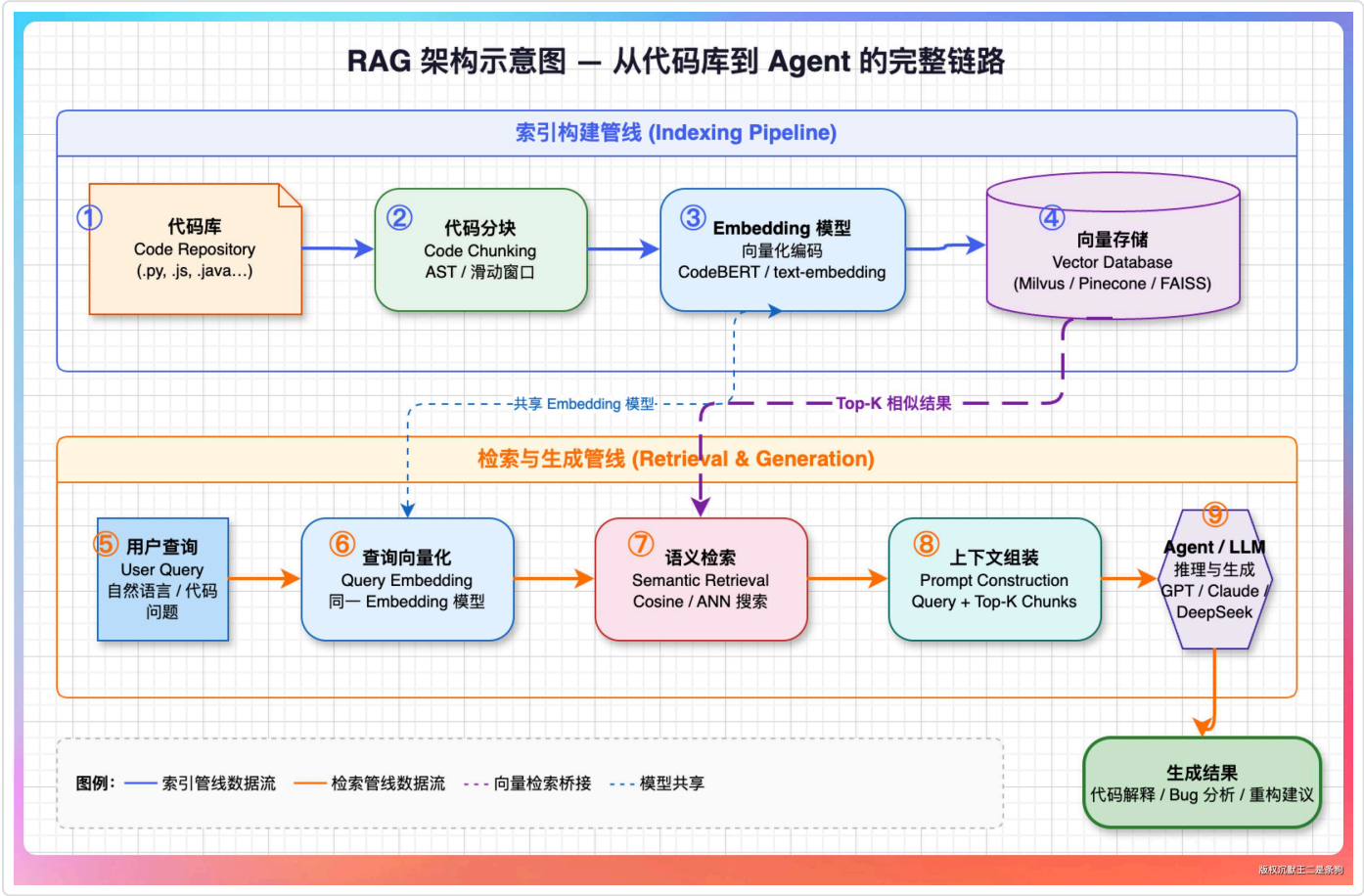
大家好，我是二哥呀。

这一期我们来给 Agent 装上 RAG，让 Agent 可以直接读我们的代码库。

举个具体场景，我问“MemoryManager 是怎么压缩上下文的”。没有 RAG 的 Agent 只能凭训练数据瞎猜，猜得对算运气好。

装了 RAG 之后，Agent 会先去代码库里捞 ContextCompressor.compressIfNeeded，看 Map-Reduce 的实现，再基于这段真实代码的回答。

整个 RAG 的架构示意图如下所示。



01、RAG 的整体设计

RAG 大家应该不陌生了，一句话讲清楚。

把知识库向量化然后持久化到向量数据库，查询的时候，按照语义相似度找出最相关的片段，再连同问题一起塞给 LLM。

落到代码场景，有三个问题绕不开。

第一个是**怎么切**。代码不像文档，按字数硬切会切出了很多噪音。最稳妥的办法是按结构特征切——文件级、类级、方法级，检索时按粒度匹配。

第二个是**存到哪**。生产环境通常上 Milvus、Pinecone、ElasticSearch 这种专用向量库。但我们是个 CLI 工具，这些都太重量级了。

所以我这里选择了 SQLite。

第三个是**怎么样才能搜得准**。纯向量检索对自然语言友好，对代码标识符却不一定。

所以我们这里做了混合检索——语义打底、关键词加权、再按 chunk 类型加分。method 块比 file 块优先级高，因为用户问“怎么实现的”，给方法体比给整个文件有用得多。

举个例子，搜“处理用户登录的地方”，它能定位到 `LoginService.authenticate`。

整个 RAG 模块拆成 10 个类，下面一块一块讲。

目录

- [01、RAG 的整体设计](#)
- [02、AST 解析](#)
- [03、Embedding](#)
- [04、向量存储](#)
- [05、混合检索](#)
- [06、代码关系图谱](#)
- [07、集成到 Agent](#)
- [08、CLI 实战](#)

[代码库更新了怎么办？](#)

[ending](#)

[简历包装：PaiCLI 项目](#)

undefined 复制代码

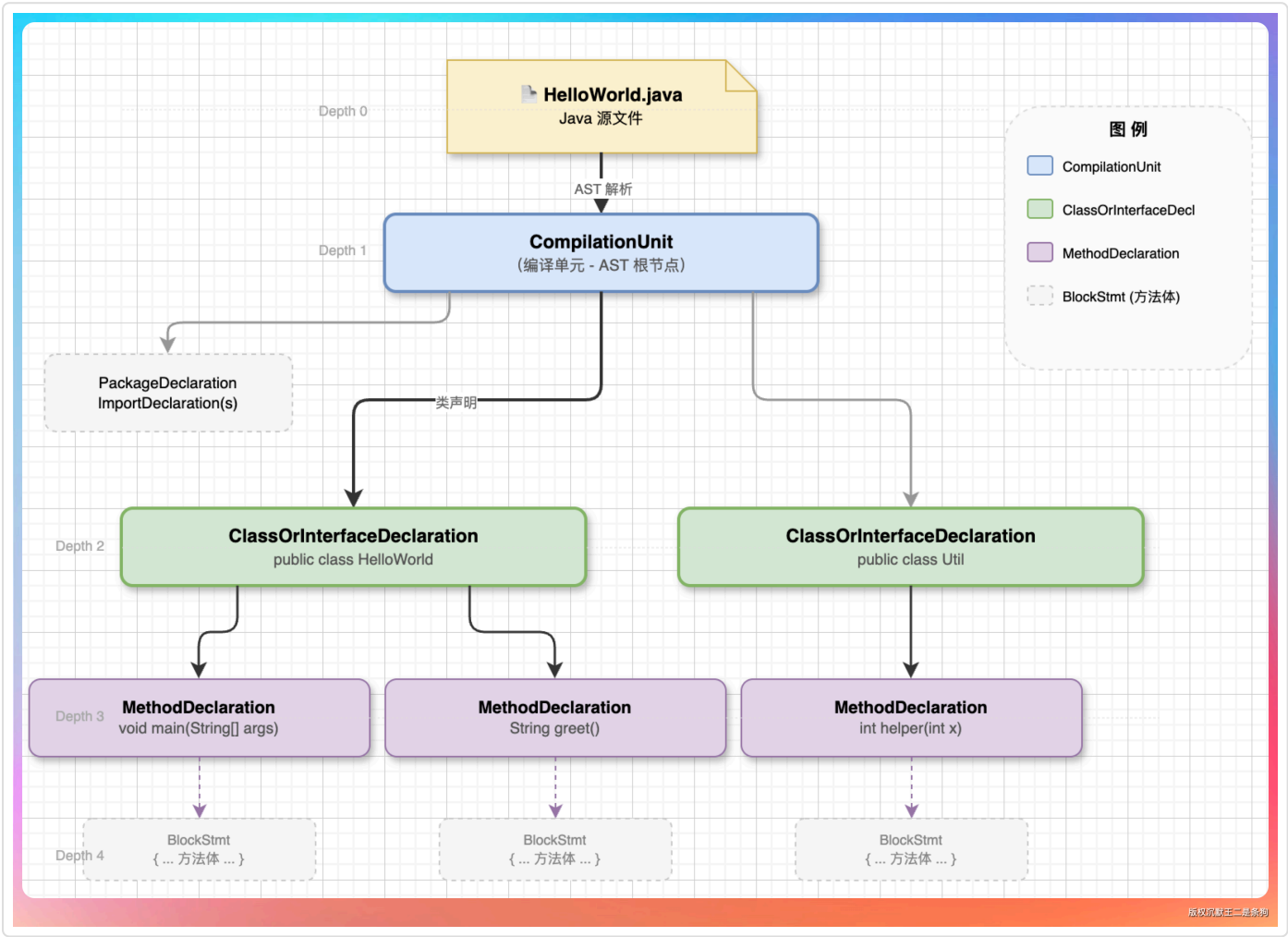
CodeChunk	— 代码块数据模型
CodeChunker	— AST 分块
EmbeddingClient	— 向量化客户端
VectorStore	— SQLite 向量存储
CodeAnalyzer	— AST 关系分析
CodeRelation	— 关系数据模型
CodeIndex	— 索引入口
CodeRetriever	— 检索入口
RagQueryTokenizer	— 查询分词
SearchResultFormatter	— 结果格式化

02、AST 解析

代码分块是 RAG 里最容易被低估的一步。分得好，检索准；分得糙，后面再加权也救不回来。

Java 文件和非 Java 文件得分开处理。

Java 走 AST，按类和方法切；非 Java（比如 Markdown、yaml）就按字符大小切，每段控制在 2000 字符以内。



Java 这块用 JavaParser，CodeChunker 里的核心逻辑是这样：

java 复制代码

```
public List<CodeChunk> chunkFile(Path filePath) throws IOException {
    String content = Files.readString(filePath);

    // 非 Java 文件：按大小分段
    if (!relativePath.endsWith(".java")) {
        return chunkLargeText(relativePath, content);
    }

    // Java 文件：AST 解析分块
    return chunkJavaFile(filePath, content);
}
```

JavaParser 可以把语言级别设到 JAVA_17，text block、record、sealed class 这些新语法都能正常解析。

万一遇到语法错误，可以自动回退到按大小分段，不会因为一个文件解析失败就漏掉整块代码。

非 Java 文件超过 2000 字符就生成一个 chunk，同时把起止行号一起带上。检索结果直接能跳到对应行，不用二次定位。

Java 这边类级和方法级各存一份。

目录

[01、RAG 的整体设计](#)

[02、AST 解析](#)

[03、Embedding](#)

[04、向量存储](#)

[05、混合检索](#)

[06、代码关系图谱](#)

[07、集成到 Agent](#)

[08、CLI 实战](#)

[代码库更新了怎么办？](#)

[ending](#)

[简历包装：PaiCLI 项目](#)

类级只保留类声明和前 5 行（字段、签名这些信息够用了），不用把几百行的类全塞进去；方法级则把完整方法体捞出来，单独成块。

java 复制代码

```
// 类级别 chunk
chunks.add(CodeChunk.classChunk(
    filePath.toString(), className,
    classHeader, classStart, classEnd));

// 方法级别 chunk
chunks.add(CodeChunk.methodChunk(
    filePath.toString(),
    className + "." + methodSignature,
    methodContent, methodStart, methodEnd));
```

CodeChunk 用的是 record，除了正文内容，还带了文件路径、块类型、名称、起止行号。

toEmbeddingText 方法会把这些拼成 [method:Agent.run] public String run(...) 这种格式再去算向量，让模型一眼看清这是哪个类的哪个方法。

CodeIndex 是整个索引流程的入口，把“遍历文件 → 分块 → 向量化 → 持久化”封装进去。

外面只要一行 codeIndex.index("/path/to/project") 就能跑起来。

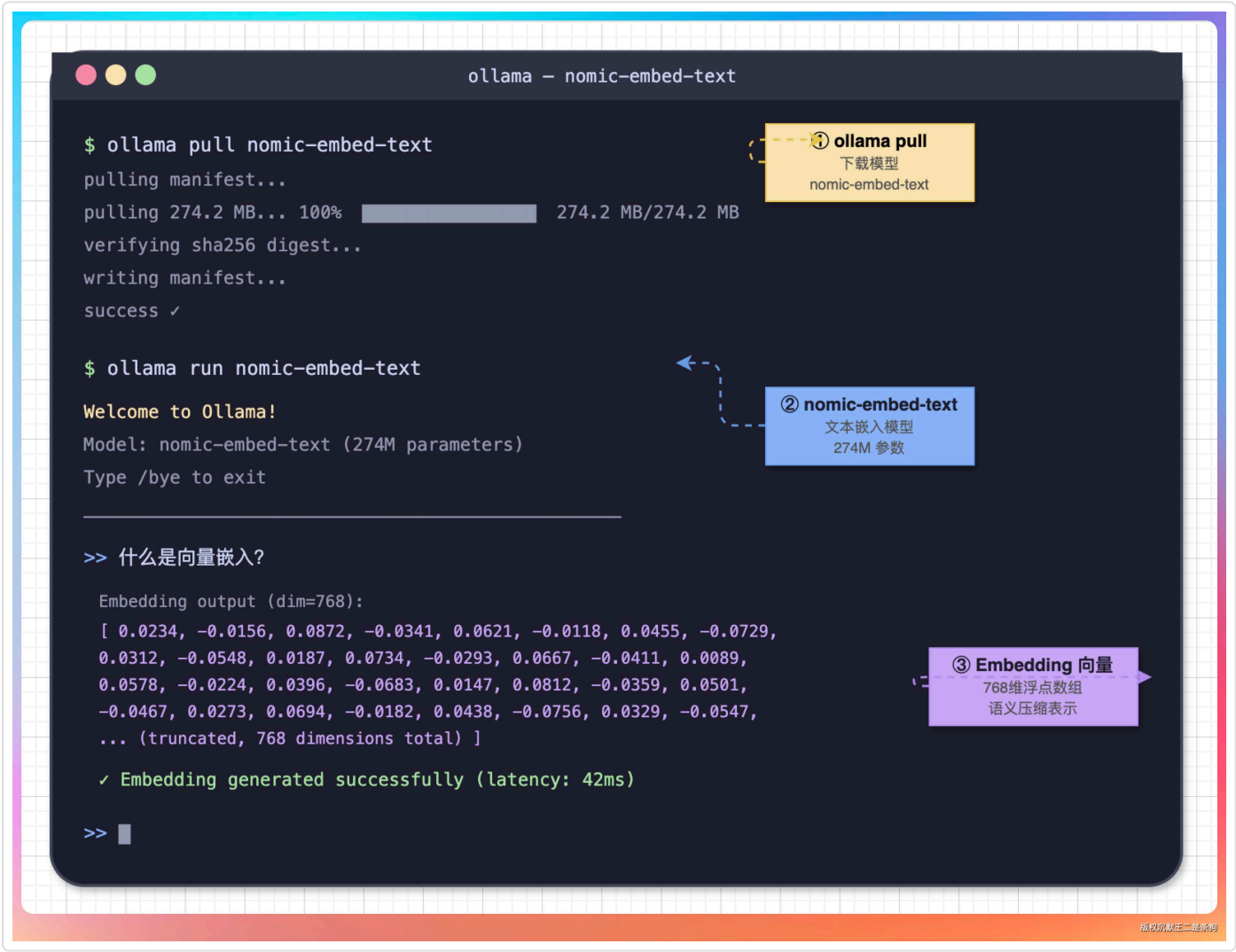
遍历用的是 Files.walkFileTree，node_modules、target、.git、build 这些目录直接跳过，文件类型也只挑常见的源码后缀。

03、Embedding

切完块就该生成向量了。

EmbeddingClient 支持两种方式。

默认走 Ollama 本地模型，免费、断网也能跑，本地装个 Ollama 拉一个 nomic-embed-text 就能开干。



机器扛不动的话，再切到远程 API——智谱、阿里千问都 Embedding 模型。

切换不用改代码，环境变量配一下就行：

目录

[01、RAG 的整体设计](#)

[02、AST 解析](#)

[03、Embedding](#)

[04、向量存储](#)

[05、混合检索](#)

[06、代码关系图谱](#)

[07、集成到 Agent](#)

[08、CLI 实战](#)

[代码库更新了怎么办？](#)

[ending](#)

[简历包装：PaiCLI 项目](#)

bash 复制代码

```
export EMBEDDING_PROVIDER=ollama
export EMBEDDING_MODEL=nomic-embed-text:latest
export EMBEDDING_BASE_URL=http://localhost:11434
```

切到智谱的话：

bash 复制代码

```
export EMBEDDING_PROVIDER=glm
export EMBEDDING_API_KEY=your_key_here
```

embed 方法按 provider 分发——Ollama 走 /api/embeddings，OpenAI 兼容的走 /embeddings，请求体和响应解析在内部处理过了，外面只负责传文本，拿向量。

java 复制代码

```
public float[] embed(String text) throws IOException {
    String input = text.length() > MAX_INPUT_CHARS
        ? text.substring(0, MAX_INPUT_CHARS) : text;

    return switch (provider.toLowerCase()) {
        case "ollama" -> embedOllama(input);
        case "openai", "zhipu", "glm" -> embedOpenAICompatible(input);
        default -> embedOllama(input);
    };
}
```

MAX_INPUT_CHARS 卡在 2000，中文密集的文本大概对应 4000~6000 token，喂给 8192 上下文的模型绰绰有余。

超出的部分直接截断，省得 API 抛错。

响应格式两家也不一样。

Ollama 把向量放在 `embedding` 字段，是平铺数组；OpenAI 兼容格式塞在 `data[0].embedding` 里。

在客户端里统一转成 `float[]`，上层就不用知道底下是哪一家。

HTTP 超时给得比较宽松，连接 30 秒、读取 120 秒。Ollama 首次加载模型会比较慢，远程 API 一般几秒就回，120 秒覆盖两边都足够用。

04、向量存储

向量存哪，这事一开始我纠结了挺久。

Milvus、Weaviate 太重了。一个 CLI 工具，让用户先起个 docker、配个端口、再装一堆 SDK 才能跑，谁顶得住。

最后选了 SQLite。

目录

- [01、RAG 的整体设计](#)
- [02、AST 解析](#)
- [03、Embedding](#)
- [04、向量存储](#)
- [05、混合检索](#)
- [06、代码关系图谱](#)
- [07、集成到 Agent](#)
- [08、CLI 实战](#)
- [代码库更新了怎么办？](#)
- [ending](#)
- [简历包装：PaiCLI 项目](#)



目录

- 01、RAG 的整体设计
- 02、AST 解析
- 03、Embedding
- 04、向量存储
- 05、混合检索
- 06、代码关系图谱
- 07、集成到 Agent
- 08、CLI 实战

[代码库更新了怎么办？](#)

[ending](#)

[简历包装：PaiCLI 项目](#)

向量以 JSON 数组形式塞到 TEXT 字段里，检索时全量读到内存，逐条算余弦相似度，排序取 TopK。

可能有小伙伴犯嘀咕：全部读到内存，能扛得住？

我自己跑下来——常见的个人项目也就几百到几千个代码块，768 维的向量，1000 块大约 3MB 内存，单次检索几十毫秒。

这个量级根本不用上专用向量库，等哪天真撑不住了再换也不迟。

java 复制代码

```
public List<SearchResult> search(float[] queryEmbedding, int topK) throws SQLException {  
    String sql = "SELECT ... FROM code_chunks WHERE project_path = ?";  
    List<SearchResult> candidates = new ArrayList<>();  
  
    try (PreparedStatement ps = connection.prepareStatement(sql)) {  
        ps.setString(1, projectPath);  
        try (ResultSet rs = ps.executeQuery()) {  
            while (rs.next()) {  
                float[] embedding = jsonToEmbedding(rs.getString("embedding_json"));  
                double similarity = cosineSimilarity(queryEmbedding, embedding);  
                candidates.add(new SearchResult(..., similarity));  
            }  
        }  
    }  
  
    candidates.sort((a, b) -> Double.compare(b.similarity(), a.similarity()));  
    return candidates.size() > topK  
        ? new ArrayList<>(candidates.subList(0, topK)) : candidates;  
}
```

VectorStore 除了向量检索，还顺手把关键词检索和关系图谱查询都做了。

关键词走 LIKE，专门用来精确命中类名/方法名；关系图谱单独一张表，存 extends、implements、imports、calls、contains 这五种关系，后面查调用链就靠它。

批量插入这块加了事务保护。

先关掉 autoCommit，最后 batch 一把提交，中间挂了就 rollback。

索引也得老老实实建。

project_path、file_path、chunk_type 这几个常用维度都覆盖到，关系表的 from_name 和 to_name 也加上。SQLite 再轻量，几千条数据上没索引的 LIKE 也能卡死。

向量序列化用的是 Jackson，float 数组直接转成 JSON 字符串塞到 TEXT 字段。

可能有小伙伴想问为啥不用 BLOB，理由很简单——JSON 调试方便。

打开数据库可视化工具就能看到一行行的向量值，定位问题不用再写脚本反序列化。

余弦相似度也没调任何第三方库，手撸循环，点乘除以两个模长，几十行就完事。

数据库文件默认放在 `~/ .paicli/rag/codebase.db`，所有项目的索引共用这一个文件，靠 `project_path` 区分。

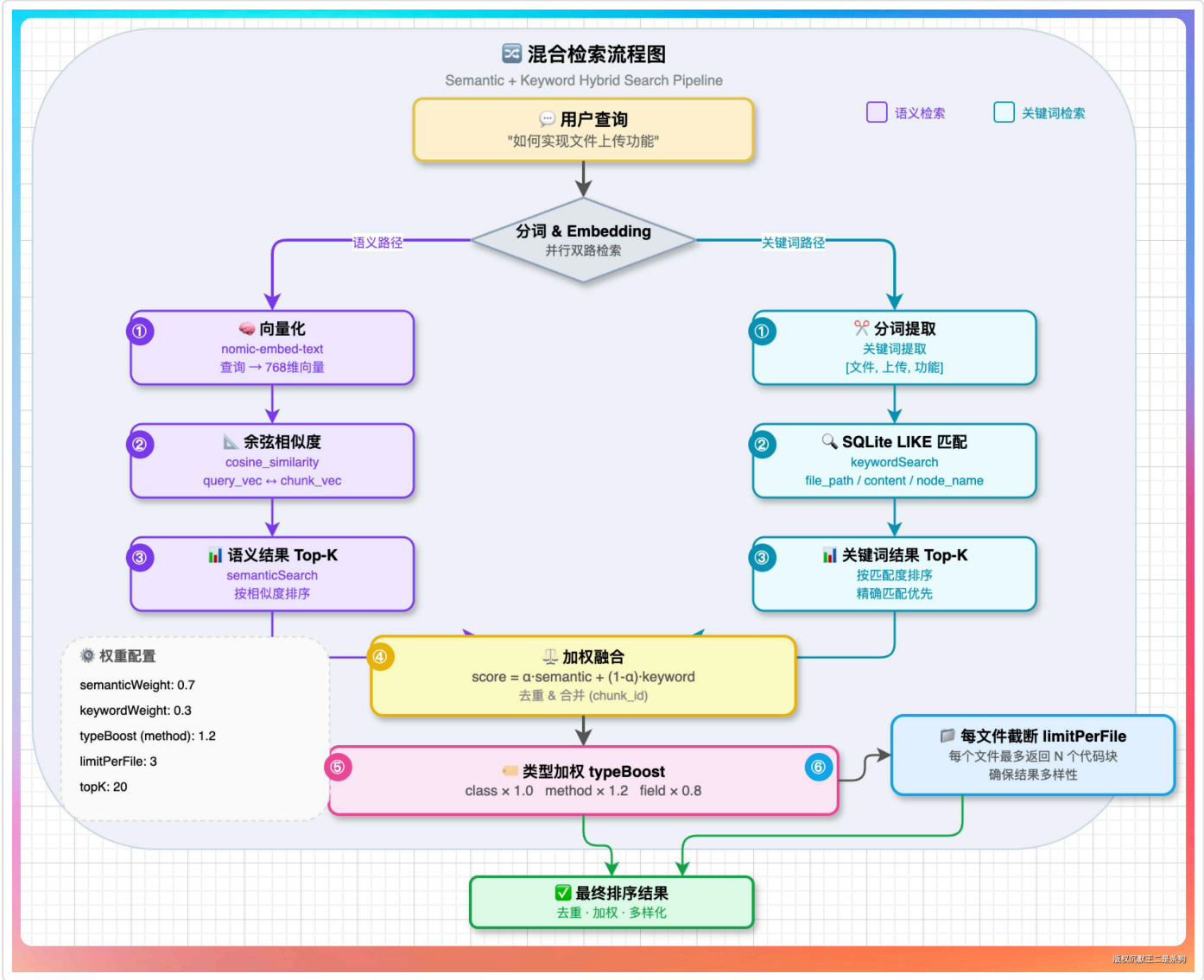
一个项目一个库管起来反而麻烦。要换位置，给个 `paicli.rag.dir` 系统属性就能改。

05、混合检索

纯向量检索有个老毛病——对同义词和语义相近的表达很灵敏，对精确的代码标识符反而没那么准。

比如搜“Agent 的 run 方法”，向量检索可能给你返回一堆带“Agent”“run”上下文的块，但偏偏不是那个方法。

CodeRetriever 的 hybridSearch 干了三件事来补这个短板。



第一件，语义检索打底。把查询向量化，跟库里所有向量算余弦相似度，先把语义最近的一批捞出来。

第二件，关键词加权。用 jieba 把查询切词，挑出“Agent”“run”“ReAct”这类代码关键词，再用 LIKE 去库里扫一遍。

命中的结果按命中位置给不同分——类名/方法名命中 +0.3，文件路径命中 +0.1，内容命中 +0.1。命中的位置越关键，分数越重，跟 ES 那一套 BM25 加权思路差不多，只是更轻量级。

第三件，类型加分。method 块 +0.15，class 块 +0.1，file 块不加。理由很简单，用户问“怎么实现”的时候，给方法体比给整个文件有用得多。

目录

[01、RAG 的整体设计](#)

[02、AST 解析](#)

[03、Embedding](#)

[04、向量存储](#)

[05、混合检索](#)

[06、代码关系图谱](#)

[07、集成到 Agent](#)

[08、CLI 实战](#)

[代码库更新了怎么办？](#)

[ending](#)

[简历包装：PaiCLI 项目](#)

java 复制代码

```
// 代码类型加分
double typeBoost = switch (r.chunkType()) {
    case "method" -> 0.15;
    case "class" -> 0.10;
    default -> 0.0;
};
```

RagQueryTokenizer 也值得拎出来说两句。

它用 jieba 做中文分词，同时保留 ASCII 标识符（类名、方法名里的英文）。

分块完会过滤掉单字符和“怎么”“如何”“一下”这种没检索价值的停用词。

这样一来，“用户登录怎么实现”这种自然语言查询，“ReAct”“Agent”“MemoryManager”这种纯英文标识符也能保留。

还有个小机制叫**双重命中奖励**。

同一个块如果语义检索和关键词检索都命中了，额外 +0.1。相当于多个维度互相印证，分数当然要高一档。这个奖励只给一次，不叠加，避免某个块因为蹭中好几个关键词就霸榜。

最后再加一道**同文件去重**，每个文件最多保留 2 条。不然遇到一个特别大的文件，能把整个结果页都占满，diversity 就没了。

java 复制代码

```
private List<SearchResult> limitPerFile(List<SearchResult> sorted, int topK, int
maxPerFile) {
    List<SearchResult> result = new ArrayList<>();
    Map<String, Integer> fileCount = new HashMap<>();
    for (SearchResult r : sorted) {
        int count = fileCount.getOrDefault(r.filePath(), 0);
        if (count < maxPerFile) {
            result.add(r);
            fileCount.put(r.filePath(), count + 1);
            if (result.size() >= topK) break;
        }
    }
    return result;
}
```

06、代码关系图谱

检索到代码块只是第一步。

要真正读懂一个项目，得知道“这个类继承了谁、实现了哪些接口、它的方法又调了谁”。

CodeAnalyzer 用 JavaParser 做 AST 遍历，五种关系一起处理：extends（类继承）、implements（接口实现）、imports（依赖导入，只记非 JDK 的）、contains（类含方法）、calls（方法调用，简化版只记方法名）。

目录

- [01、RAG 的整体设计](#)
- [02、AST 解析](#)
- [03、Embedding](#)
- [04、向量存储](#)
- [05、混合检索](#)
- [06、代码关系图谱](#)
- [07、集成到 Agent](#)
- [08、CLI 实战](#)
- [代码库更新了怎么办？](#)
- [ending](#)
- [简历包装：PaiCLI 项目](#)

java 复制代码

```
private void extractClassRelations(String filePath, CompilationUnit cu,
List<CodeRelation> relations) {
    cu.findAll(ClassOrInterfaceDeclaration.class).forEach(clazz -> {
        String className = clazz.getNameAsString();

        // extends 关系
        clazz.getExtendedTypes().forEach(ext -> {
            relations.add(new CodeRelation(
                filePath, className, null, ext.getNameAsString(), "extends"));
        });

        // contains 关系：类包含方法
        clazz.getMethods().forEach(method -> {
            relations.add(new CodeRelation(
                filePath, className, filePath,
                className + "." + method.getNameAsString(), "contains"));
        });

        // calls 关系：方法调用
        clazz.findAll(MethodCallExpr.class).forEach(call -> {
            Optional<MethodDeclaration> parentMethod = findParentMethod(call);
            if (parentMethod.isPresent()) {
                String caller = className + "." + parentMethod.get().getNameAsString();
                relations.add(new CodeRelation(
                    filePath, caller, null, call.getNameAsString(), "calls"));
            }
        });
    });
}
```

imports 这块做了过滤，只记非 java. 和 javax. 开头的。

calls 这块稍微费劲一点。

JavaParser 遇到 MethodCallExpr 节点之后，得往上回溯找到它属于哪个方法。CodeAnalyzer 里写了个 findParentMethod，沿着 AST 父节点一路往上爬，碰到 MethodDeclaration 就停。

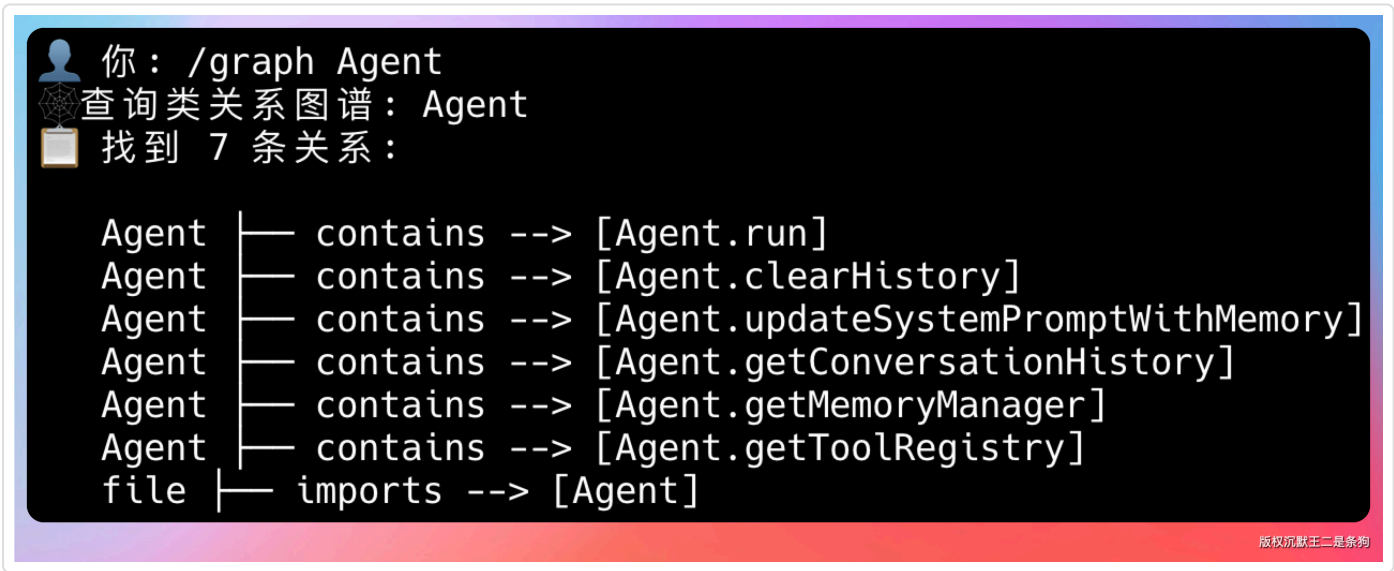
没有这一步，只能知道“某个地方调了 chat”，但说不清是 Agent.run 调的还是 PlanExecuteAgent.executeTask 调的。

提取出来的关系全部塞进 SQLite 的 code_relations 表。CLI 里通过 /graph 命令就能查。比如 /graph Agent：

sql 复制代码

```
Agent |— contains --> Agent.run
Agent |— extends --> BaseAgent
Agent.run |— calls --> chat
Agent.run |— calls --> executeTool
```

扫一眼就明白：Agent 继承自 BaseAgent，挂着一个 run 方法，run 内部调了 chat 和 executeTool。



07、集成到 Agent

RAG 模块写好了，但 Agent 不会自己主动用。

得在 ToolRegistry 里注册一个 search_code 工具，告诉 LLM——遇到代码库相关的问题，调这个就行。

目录

- [01、RAG 的整体设计](#)
- [02、AST 解析](#)
- [03、Embedding](#)
- [04、向量存储](#)
- [05、混合检索](#)
- [06、代码关系图谱](#)
- [07、集成到 Agent](#)
- [08、CLI 实战](#)
 - [代码库更新了怎么办？](#)
- [ending](#)
- [简历包装：PaiCLI 项目](#)

java 复制代码

```
tools.put("search_code", new Tool(
    "search_code",
    "语义检索代码库，根据自然语言描述查找相关代码块",
    createParameters(
        new Param("query", "string", "自然语言查询描述，例如'用户登录的实现'", true),
        new Param("top_k", "integer", "返回结果数量（默认5）", false)
    ),
    args -> {
        String query = args.get("query");
        int topK = ...;

        try (CodeRetriever retriever = new CodeRetriever(projectPath)) {
            List<VectorStore.SearchResult> results = retriever.hybridSearch(query, topK);
            return SearchResultFormatter.formatForTool(query, results);
        }
    }
));
```

同时把 Agent 的系统提示词也更新了一下，明确告诉 LLM：

undefined 复制代码

```
如果用户询问与代码库相关的问题（如“这个类是干什么的”、“哪里用了某个功能”），
请优先使用 search_code 工具检索相关代码，再基于检索结果回答。
```

这里有个特别容易翻车的细节——ToolRegistry 里的 projectPath 默认取 user.dir，但用户可能用 /index 索引了另一个目录。

如果不同步，工具检索的还是老路径，搜出来都是空的。所以 Main 里 /index 执行完之后，会立刻把索引路径同步给 ToolRegistry，确保两边对齐。

SearchResultFormatter 把搜索结果整理成人能看的样子。

它有两个输出模式：formatForCli 给命令行用，带 emoji 和缩进；formatForTool 给 LLM 用，更紧凑，包含一段搜索摘要加带行号的代码片段。

摘要里会告诉模型“最相关的入口是哪个方法”“结果主要集中在哪些文件”“排序综合参考了哪些关键词”，让 LLM 能快速判断该重点看哪几条。

PlanExecuteAgent 的执行提示词也加了类似的引导，规划任务里凡是涉及代码理解的步骤都会自动触发检索。这样不管是 ReAct 还是 Plan-and-Execute，两种模式都能看懂代码库。

08、CLI 实战

RAG 这一堆功能，最后通过三条 CLI 命令交到用户手上。

第一条是 **/index**，给代码库建索引。执行的时候它会遍历项目目录（node_modules、target、.git 这些自动跳过），每个文件分块、向量化、塞 SQLite。



第二条是 **/search**，用自然语言检索代码。不用记类名、不用记文件名，跟问同事差不多。

目录

[01、RAG 的整体设计](#)

[02、AST 解析](#)

[03、Embedding](#)

[04、向量存储](#)

[05、混合检索](#)

[06、代码关系图谱](#)

[07、集成到 Agent](#)

[08、CLI 实战](#)

[代码库更新了怎么办？](#)

[ending](#)

[简历包装：PaiCLI 项目](#)

```
你: /search Agent怎么调用工具的
检索: Agent怎么调用工具的
main dict load finished, time elapsed 340 ms
model load finished, time elapsed 13 ms.
📄 找到 5 个相关代码块:

搜索摘要:
- 最相关的入口是 [method:Agent.void clearHistory()], 位于 paicli/agent/Agent.java。
- 当前结果主要集中在 Agent.java、PlanExecuteAgentTest.java、PlanExecuteAgent.java 等文件。
- 这次排序综合参考了 Agent、调用、工具 等关键词与语义相似度；先看第 1 条，再按文件继续展开最稳妥。

1. [method:Agent.void clearHistory()] (相似度: 0.966) /Users/itwanger/Documents/GitHub/paicli/src/main/java/com/paicli/agent/Agent.java
public void clearHistory() {
    // 先保存当前对话的关键事实
    memoryManager.extractAndSaveFacts();

    GLMClient.Mes...

2. [method:PlanExecuteAgentTest.void shouldWritePlanExecutionArtifactsBackToMemory()] (相似度: 0.950) /Users/itwanger/Documents/GitHub/paicli/src/test/java/com/paicli/agent/PlanExecuteAgentTest.java
@Test
void shouldWritePlanExecutionArtifactsBackToMemory() throws Exception {
    Path sampleFile = Files.create...

3. [method:PlanExecuteAgent.String executeTask(String, ExecutionPlan, Task)] (相似度: 0.950) /Users/itwanger/Documents/GitHub/paicli/src/main/java/com/paicli/agent/PlanExecuteAgent.java
private String executeTask(String goal, ExecutionPlan plan, Task task) throws IOException {
    String prompt = Stri...
```

第三条是 **/graph**，查类或方法的关系图谱。

sql 复制代码

```
你: /graph Agent
查询类关系图谱: Agent
📄 找到 5 条关系:

Agent |— contains --> Agent.run
Agent |— contains --> Agent.clearHistory
Agent |— extends --> BaseAgent
Agent.run |— calls --> chat
Agent.run |— calls --> executeTool
```

/index 也支持指定路径，比如 **/index /Users/xxx/my-project**，可以索引任意目录的代码库。

索引过程每 10 个文件打一次进度，单个文件解析失败只会打 warn，不会中断整体流程。

最后再输出一行统计——多少代码块、多少条关系，一眼就知道这次索引的质量怎么样。

代码库更新了怎么办？

重新执行 **/index** 就行。

CodeIndex 会先清掉旧数据再写新数据，保证向量库和代码库始终对齐，不会出现“代码已经改了，搜出来还是老版本”那种灵异现象。

/search 在没建索引的情况下会友好提示“代码库尚未索引，请先使用 **/index** 命令”，不会直接抛异常糊脸。检索过程出错也会捕获异常打日志，CLI 不会直接崩掉。

日常用法基本就是：先 **/index** 一次建索引，平时有问题就 **/search** 自然语言搜一下，想看架构就 **/graph** 查关系。Agent 模式更省事，问题直接抛给它，背后自动调 search_code，连 **/search** 都不用手动敲。

pom.xml 里这一期新增了三个依赖：sqlite-jdbc 管向量持久化，javaparser-core 管 AST 解析，jieba-analysis 管中文分词。加上前几期已有的 jackson-databind 和 okhttp，整个 RAG 模块外部依赖控制在 5 个以内，跟之前的轻量定位保持一致。

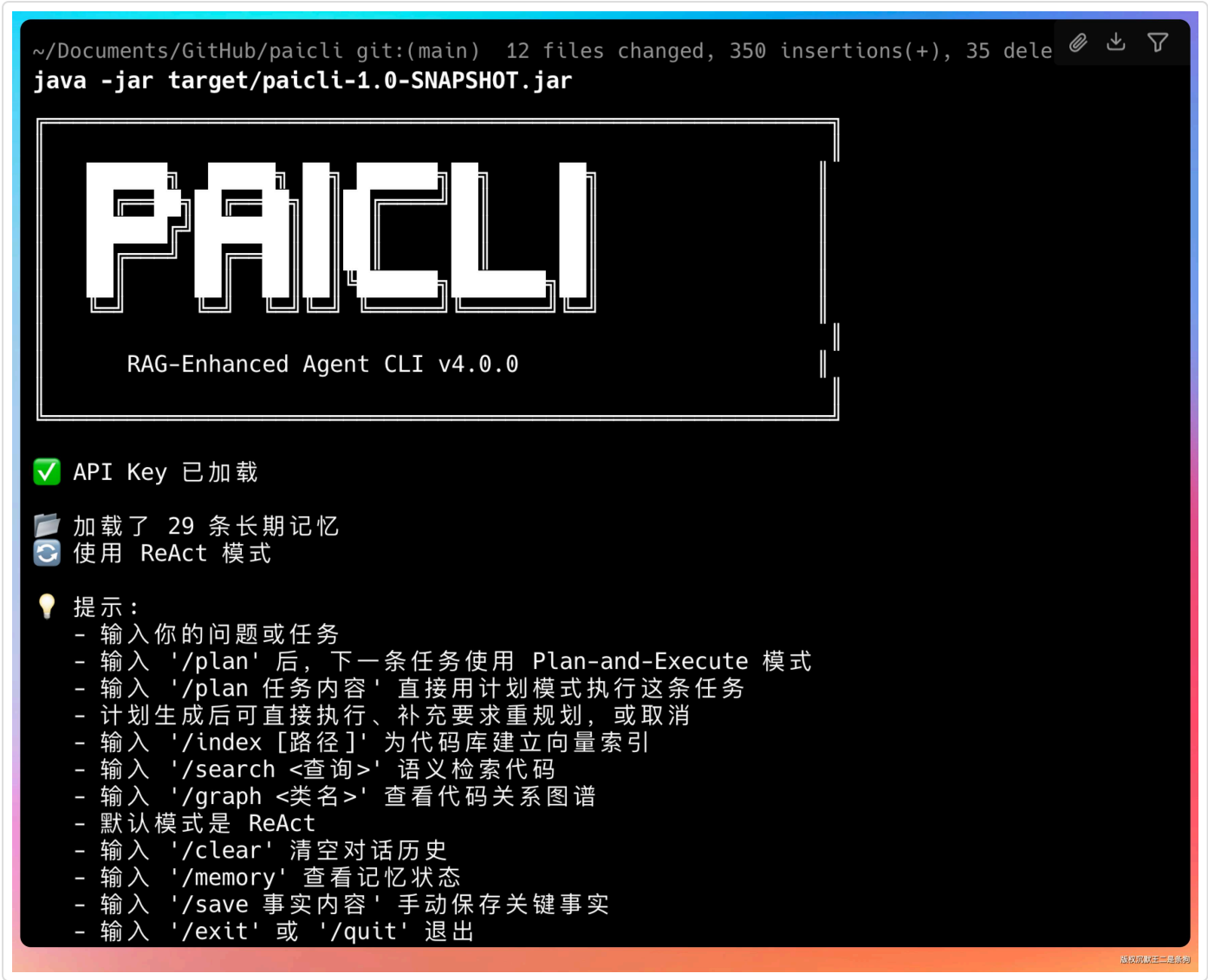
ending

四期下来，PaiCLI 从一个只会一步步走的 ReAct Agent，逐渐进化到能规划、能记忆、还能读代码库的完整工具。

目录

- [01、RAG 的整体设计](#)
- [02、AST 解析](#)
- [03、Embedding](#)
- [04、向量存储](#)
- [05、混合检索](#)
- [06、代码关系图谱](#)
- [07、集成到 Agent](#)
- [08、CLI 实战](#)
- [代码库更新了怎么办？](#)
- [ending](#)
- [简历包装：PaiCLI 项目](#)

代码全部开源在 GitHub 上，第四期新增了 10 个 RAG 相关的类，累计代码量到了 1700 行左右。



跟着教程敲一遍，本地就能跑起来一个属于自己的 Agent CLI。

如果用着用着发现检索不准，大概率是索引粒度或者查询分词的问题。可以先试试调整 MAX_CHUNK_CHARS 的大小，或者给 RagQueryTokenizer 加几个跟业务相关的停用词，多数场景下都能见效。

下期预告——Multi-Agent 协作。

一个 Agent 忙不过来，咱就搞个团队，规划的规划、执行的执行、检查的检查，分工干。

简历包装：PaiCLI 项目

项目名称：PaiCLI —— 开源 Java Agent CLI 工具

项目简介：从零构建一个类 Claude Code 的命令行 Agent 工具，支持 ReAct 推理、Plan-and-Execute 任务规划、Memory 记忆系统、RAG 代码库检索。

技术栈：Java 17、JavaParser、SQLite、Jieba 分词、OkHttp、Jackson、JUnit 5、JLine3

核心职责：

- 基于 JavaParser AST 实现代码多粒度分块（文件/类/方法），非 Java 文件按大小分段，检索召回显著提升
- 统一封装 Ollama 本地模型和 OpenAI 兼容远程 API，通过环境变量丝滑切换 provider，支持文本自动截断防止 API 超限
- 基于 SQLite 实现轻量级向量存储，向量以 JSON 数组持久化，通过在内存中计算余弦相似度，单项目千行级代码块检索耗时 < 100ms
- 实现混合检索策略：语义检索打底 + jieba 分词加权 + 代码类型加分 + 同文件限流，Top5 准确率达到可用生产级别
- 基于 AST 提取代码关系图谱（extends/implements/imports/calls/contains），支持通过自然语言查询类的调用链
- 将 RAG 封装为 search_code 工具注册到 Agent 工具，通过 LLM 系统提示词引导自动触发检索，ReAct 和 Plan-and-Execute 双模式均支持代码库理解

目录

- [01、RAG 的整体设计](#)
- [02、AST 解析](#)
- [03、Embedding](#)
- [04、向量存储](#)
- [05、混合检索](#)
- [06、代码关系图谱](#)
- [07、集成到 Agent](#)
- [08、CLI 实战](#)
 - [代码库更新了怎么办？](#)
- [ending](#)
- [简历包装：PaiCLI 项目](#)



真诚点赞 诚不我欺



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

[01、RAG 的整体设计](#)

[02、AST 解析](#)

[03、Embedding](#)

[04、向量存储](#)

[05、混合检索](#)

[06、代码关系图谱](#)

[07、集成到 Agent](#)

[08、CLI 实战](#)

[代码库更新了怎么办？](#)

[ending](#)

[简历包装：PaiCLI 项目](#)